

10. Estrategia de backup y recuperación

VentaGamer implementa la funcionalidad opcional de nivel avanzado de respaldo de información: generación de **copias de seguridad de la base de datos** desde la propia aplicación, historial de respaldos y **verificación de integridad** de los datos críticos. Este documento responde las tres preguntas que exige la consigna: qué datos se respaldan, con qué frecuencia, y cómo se recupera la información ante fallos.

10.1. Qué datos se respaldan

El backup abarca la **base de datos completa** (`VentaGamer` en SQL Server), lo que incluye:

Área	Tablas	Criticidad
Seguridad	Users, Roles, Permissions, RolePermissions, RoleHierarchies	Alta — cuentas y esquema de acceso
Ventas	Orders, OrderItems	Máxima — registro contable del negocio, irrecuperable si se pierde
Catálogo	Products	Alta — aunque puede reconstruirse, con costo
Operación	Carts, CartItems	Baja — datos transitorios
Auditoría	AuditLogs	Alta — trazabilidad y cumplimiento
Configuración	SystemSettings, Languages, Translations	Media — reconstruible desde el seed
Chatbot	AiConversations, AiMessages	Media — datos personales de los usuarios

Se respalda la base completa (y no tablas sueltas) porque garantiza la **consistencia referencial**: un respaldo parcial podría restaurar pedidos que referencian usuarios o productos inexistentes.

10.2. Cómo se genera el backup

Desde la pantalla `/maintenance` (permiso `backup.manage`), el usuario técnico ejecuta la creación del respaldo. El backend (`MaintenanceService`) lanza un comando nativo de SQL Server:

```
BACKUP DATABASE [VentaGamer]
TO DISK = '/var/opt/mssql/data/backups/VentaGamer_{fecha}.bak'
WITH COMPRESSION;
```

Puntos clave:

- El archivo `.bak` es un **backup completo y comprimido**, el formato estándar de SQL Server.

- Se escribe dentro del **volumen Docker** `sqldata`, que persiste aunque el contenedor se destruya y recree.
- La pantalla muestra el **historial de respaldos** (fecha, tamaño, archivo), consultado del catálogo interno del motor (`msdb.dbo.backupset`), por lo que refleja también los backups realizados por fuera de la aplicación.
- Toda la operatoria queda auditada en la bitácora (quién generó cada backup y cuándo).

10.3. Política de frecuencia propuesta

Para un despliegue productivo de un comercio de este tipo se propone el siguiente esquema, escalonado según la criticidad de los datos:

Tipo de respaldo	Frecuencia	Justificación
Backup completo (el implementado)	Diario, fuera del horario pico (por ejemplo, 3:00)	Restaura el sistema entero a un punto conocido.
Backup diferencial	Cada 6 horas	Reduce la pérdida máxima de datos a horas, con archivos chicos.
Backup del log de transacciones	Cada 15-30 minutos	Permite recuperación a un punto en el tiempo (por ejemplo, justo antes de un error).
Copia fuera del servidor	Diaria, tras el backup completo	Un respaldo que vive en el mismo disco que la base no protege contra la falla de ese disco: debe copiarse a otro equipo o almacenamiento.
Prueba de restauración	Mensual	Un backup solo es válido si se demostró que restaura: se verifica en un entorno de prueba.

Retención sugerida: 7 backups diarios, 4 semanales y 12 mensuales, eliminando los anteriores.

En el entorno de demostración del trabajo práctico, el backup se dispara **a demanda** desde la interfaz; la programación automática se resolvería con un programador de tareas (cron en el host o SQL Server Agent) invocando el mismo comando.

10.4. Procedimiento de recuperación ante fallos

La aplicación genera y lista los backups; la **restauración es un procedimiento manual y deliberado** del administrador técnico. Es una decisión de diseño: restaurar una base sobrescribe todos los datos actuales, por lo que no conviene exponerlo como un botón en la interfaz web.

Procedimiento documentado (con el sistema en Docker):

```
# 1. Detener la API para que no haya conexiones activas contra la base
docker compose stop backend

# 2. Listar los backups disponibles dentro del contenedor de SQL Server
docker exec ventagamer-sql ls /var/opt/mssql/data/backups/

# 3. Restaurar el backup elegido (sobrescribe la base actual)
docker exec ventagamer-sql /opt/mssql-tools18/bin/sqlcmd \
-S localhost -U sa -P "$MSSQL_SA_PASSWORD" -C \
-Q "RESTORE DATABASE [VentaGamer] \
FROM DISK = '/var/opt/mssql/data/backups/VentaGamer_2026xxx.bak' \
WITH REPLACE;"

# 4. Volver a levantar la API (aplica migraciones pendientes si las hubiera)
docker compose start backend
```

Escenarios de falla cubiertos:

Escenario	Recuperación
Error humano (borrado o modificación masiva incorrecta)	Restaurar el último backup completo previo al error.
Corrupción de la base	Restaurar backup; el esquema se revalida con las migraciones al arrancar la API.
Pérdida del contenedor o del volumen	Recrear los contenedores con <code>docker compose up</code> ; restaurar el <code>.bak</code> desde la copia externa.
Migración de servidor	Copiar el <code>.bak</code> al nuevo host, levantar el stack y restaurar.

10.5. Verificación de integridad (HMAC-SHA256)

Complementando el respaldo, el sistema ofrece una **verificación de integridad** de los datos críticos (permiso `integrity.check`, misma pantalla de mantenimiento):

- El servicio recorre las tablas **Users** y **Orders** y calcula para cada registro una firma **HMAC-SHA256** sobre una representación canónica de sus campos (por ejemplo, para un pedido: número, usuario, total y fecha).
- HMAC combina la función de hash SHA-256 con una **clave secreta**: sin la clave, no es posible generar firmas válidas. Esto permite detectar modificaciones hechas por fuera de la aplicación (por ejemplo, un UPDATE directo en la base alterando el total de un pedido).
- El resultado es un reporte por tabla con las firmas calculadas y las anomalías detectadas.

Alcance actual y mejora futura: en esta versión el mecanismo calcula y demuestra las firmas en el momento de la verificación. El paso siguiente natural es **almacenar la firma junto a cada registro** al crearlo, de modo que la verificación compare firma almacenada contra firma recalculada y detecte cualquier alteración posterior de manera concluyente.

10.6. Otras capas de protección de datos

La estrategia de respaldo se apoya en decisiones de diseño que reducen la probabilidad de necesitarla:

- **Baja lógica de productos:** eliminar un producto no borra datos, solo lo desactiva; el historial de ventas queda intacto.
- **Snapshot en pedidos:** los ítems de un pedido copian título y precio al momento de la compra; el registro contable no depende del estado actual del catálogo.
- **Transacciones en checkout:** imposible llegar a estados intermedios inconsistentes.
- **Políticas de borrado cuidadas:** los pedidos no se borran en cascada; la bitácora sobrevive a la eliminación de usuarios (detalle en [05 — Base de datos](#)).
- **Volúmenes Docker persistentes:** los datos de SQL Server (`sqldata`) y los modelos de IA (`ollamadata`) sobreviven a la recreación de los contenedores.